

TASK 05 — Swagger/OpenAPI & Professional API Documentation

Step 1 — Understand OpenAPI

OpenAPI is a standard way to describe a REST API.

Example, for your **Product API**:

GET /api/products

OpenAPI can document that this endpoint returns a list of products.

Swagger UI is a web interface that displays your OpenAPI documentation in an easy-to-use format.

Example,

GET /api/products

POST /api/products

GET /api/products/{id}

PUT /api/products/{id}

DELETE /api/products/{id}

You can expand an endpoint and use **Try it out** to send a request directly from the browser.

So:

OpenAPI = API description/specification

Swagger UI = User interface that displays the OpenAPI documentation

API Specification

The **API specification** is the actual description of your API.

It contains information such as:

openapi: 3.0.0

info:

title: Product API

version: 1.0.0

paths:

/api/products:

get:

summary: Get all products

This tells tools what your API looks like.

Endpoint Documentation

An **endpoint** is a specific URL through which clients communicate with your API.

Example:

GET /api/products

Documentation should explain:

Item	Example
Method	GET
URL	/api/products
Purpose	Get all products
Response	List of products

Success status 200

Another example:

GET /api/products/1

This gets the product whose ID is 1.

Request Schema

A **request schema** describes the data that the client sends to your API.

For example, when creating a product:

POST /api/products

Request body:

```
{  
  "name": "Laptop",
```

```
"price": 55000
}
```

The schema defines what fields are expected:

name → String

price → Number

For example:

```
public class ProductRequest {
    private String name;
    private double price;
}
```

Response Schema

A **response schema** describes the data returned by the API.

For example:

```
{
    "id": 1,
    "name": "Laptop",
    "price": 55000
}
```

The response schema tells us:

id → Integer

name → String

price → Number

Step 2 — Add OpenAPI Dependency

The goal is to add **Swagger/OpenAPI support** to your Spring Boot project.

If you are using **Maven**, add this dependency to pom.xml:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.8.13</version>
</dependency>
```

Then save the pom.xml.

Maven will download the required OpenAPI/Swagger libraries.

After starting your Spring Boot application, Swagger UI is normally available at:

<http://localhost:8083/swagger-ui.html>

or:

<http://localhost:8083/swagger-ui/index.html>

If your Product Service runs on another port, replace 8083 with that port.

Step 3 — Document Every Endpoint

POST — Create Product

POST /api/v1/products

Purpose:

Create a new product.

Request:

```
{
  "name": "Laptop",
  "price": 55000
}
```

Response:

```
{
  "id": 1,
  "name": "Laptop",
```

```
"price": 55000  
}
```

Usually the response status is:

201 Created

GET — Get All Products

GET /api/v1/products

Purpose:

Retrieve all products.

Response:

```
[  
  {  
    "id": 1,  
    "name": "Laptop",  
    "price": 55000  
  },  
  {  
    "id": 2,  
    "name": "Mobile",  
    "price": 25000  
  }  
]
```

Status:

200 OK

GET — Get Product by ID

GET /api/v1/products/{id}

Example:

GET /api/v1/products/1

Here:

{id} = 1

Response:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 55000  
}
```

If the product doesn't exist:

404 Not Found

PUT — Update Entire Product

PUT /api/v1/products/{id}

Example:

PUT /api/v1/products/1

Request:

```
{  
  "name": "Gaming Laptop",  
  "price": 75000  
}
```

PUT generally means:

Replace/update the complete resource.

PATCH — Partially Update Product

PATCH /api/v1/products/{id}

Example:

PATCH /api/v1/products/1

Request:

```
{  
  "price": 70000  
}
```

PATCH means:

Update only the fields that are provided.

For example, the existing product:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 55000  
}
```

After PATCH:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 70000  
}
```

The name did not change.

DELETE — Delete Product

DELETE /api/v1/products/{id}

Example:

DELETE /api/v1/products/1

Purpose:

Delete product ID 1.

A common successful response is:

204 No Content

Step 4 — Document Request Parameters

Path Parameter

A path parameter is part of the URL.

Example:

GET /api/v1/products/{id}

Here:

{id}

is a **path parameter**.

Spring Boot:

```
@GetMapping("/{id}")  
  
public Product getProductById(@PathVariable Long id) {  
    return productService.getProductById(id);  
}
```

The important part is:

@PathVariable Long id

Example request:

GET /api/v1/products/10

Here:

id = 10

Query Parameter

Query parameters come after.

Example:

GET /api/v1/products?name=Laptop

Here:

name=Laptop

is a query parameter.

Spring Boot:

@GetMapping

```
public List<Product> getProducts(  
    @RequestParam(required = false) String name) {  
  
    return productService.searchProducts(name);  
}
```

The following is optional:

required = false

So both requests can work:

GET /api/v1/products

and:

GET /api/v1/products?name=Laptop

Request Body

Request body contains JSON data sent to the server.

For POST:

POST /api/v1/products

Body:

```
{  
    "name": "Laptop",  
    "price": 55000  
}
```

Spring Boot:

@PostMapping

```
public Product createProduct(  
    @RequestBody Product product) {  
    return productService.createProduct(product);  
}
```

The important annotation is:

`@RequestBody`

It tells Spring:

Convert the incoming JSON into a Java object.

Required and Optional Fields

Suppose your Product has:

name

price

description

You may decide:

name → Required

price → Required

description → Optional

You can use validation annotations.

Example:

```
public class ProductRequest {  
    @NotBlank  
    private String name;  
    @NotNull  
    @Positive  
    private Double price;  
    private String description;
```

```
}
```

Meaning:

@NotBlank



name must be provided

@NotNull



price cannot be null

@Positive



price must be greater than 0

Complete Controller Example

```
@RestController
```

```
@RequestMapping("/api/v1/products")
```

```
public class ProductController {
```

```
    @PostMapping
```

```
    public Product createProduct(
```

```
        @RequestBody Product product) {
```

```
        return productService.createProduct(product);
```

```
    }
```

```
    @GetMapping
```

```
    public List<Product> getAllProducts() {
```

```
        return productService.getAllProducts();
```

```
    }
```

```
    @GetMapping("/{id}")
```

```
public Product getProductById(  
    @PathVariable Long id) {  
    return productService.getProductById(id);  
}  
  
@PutMapping("/{id}")  
public Product updateProduct(  
    @PathVariable Long id,  
    @RequestBody Product product) {  
    return productService.updateProduct(id, product);  
}  
  
@PatchMapping("/{id}")  
public Product partialUpdateProduct(  
    @PathVariable Long id,  
    @RequestBody Product product) {  
    return productService.partialUpdateProduct(id, product);  
}  
  
@DeleteMapping("/{id}")  
public void deleteProduct(  
    @PathVariable Long id) {  
    productService.deleteProduct(id);  
}  
}
```

Step 5 — Document Responses

200 — OK

Means:

The request was successful.

Example:

GET /api/v1/products

Response:

```
[  
  {  
    "id": 1,  
    "name": "Laptop",  
    "price": 55000  
  }  
]
```

Status:

200 OK

Usually used for successful GET, PUT, or PATCH requests that return data.

201 — Created

Means:

A new resource was successfully created.

Example:

POST /api/v1/products

Request:

```
{  
  "name": "iPhone 17",  
  "price": 80000  
}
```

```
}
```

Response:

```
{
```

```
  "id": 1,
```

```
  "name": "iPhone 17",
```

```
  "price": 80000
```

```
}
```

Status:

201 Created

POST → commonly 201 when creation succeeds.

204 — No Content

Means:

The request succeeded, but there is no response body to return.

Example:

DELETE /api/v1/products/1

Response:

204 No Content

There is no JSON response.

DELETE

↓

Product deleted

↓

204 No Content

400 — Bad Request

Means:

The client sent invalid data.

For example:

```
{  
  "name": "",  
  "price": -500  
}
```

If your API requires a non-empty name and positive price, the request is invalid.

Response:

400 Bad Request

You can return an error message such as:

```
{  
  "message": "Price must be greater than 0"  
}
```

404 — Not Found

Means:

The requested product does not exist.

Example:

GET /api/v1/products/999

If product 999 doesn't exist:

404 Not Found

Response:

```
{  
  "message": "Product not found"  
}
```

409 — Conflict

Means:

The request conflicts with the current state of the system.

For example, suppose product names must be unique.

You already have:

iPhone 17

A client tries to create another:

```
{  
  "name": "iPhone 17",  
  "price": 80000  
}
```

Your API could return:

409 Conflict

Response:

```
{  
  "message": "Product already exists"  
}
```

500 — Internal Server Error

Means:

Something unexpected went wrong on the server.

For example:

Database connection failure

Unexpected Java exception

Server configuration problem

Response:

500 Internal Server Error

You generally don't expose sensitive internal error details to the client.

Easy Table

Status	Meaning	Example
200	Success	Get product
201	Created	Create product
204	Success, no body	Delete product
400	Invalid request	Invalid price
404	Resource not found	Product doesn't exist
409	Conflict	Duplicate product
500	Server error	Database/server failure

Important

You don't necessarily use **all seven statuses for every endpoint**.

You document the statuses that are **applicable to that endpoint**.

Step 6 — Add Request Examples

POST Request

Endpoint:

POST /api/v1/products

Request body:

```
{  
  "name": "iPhone 17",  
  "description": "Smartphone",  
  "price": 80000,  
  "category": "Mobile",  
  "stock": 25  
}
```

Meaning:

name → Product name

description → Product description

price → Product price

category → Product category

stock → Available quantity

Java Product Class

Your Java model could look like:

```
public class Product {  
    private Long id;  
    private String name;  
    private String description;  
    private Double price;  
    private String category;  
    private Integer stock;  
    // getters and setters  
}
```

Then Swagger/OpenAPI can use this model to describe the request body.

Step 7 — Add Response Examples

Successful Response Example

For:

POST /api/v1/products

Request:

```
{  
    "name": "iPhone 17",  
    "description": "Smartphone",  
    "price": 80000,
```

```
"category": "Mobile",  
"stock": 25  
}
```

Successful response:

```
{  
  "id": 1,  
  "name": "iPhone 17",  
  "description": "Smartphone",  
  "price": 80000,  
  "category": "Mobile",  
  "stock": 25  
}
```

Status:

201 Created

Failure Response Example

Suppose the price is invalid:

```
{  
  "name": "iPhone 17",  
  "price": -500  
}
```

Response:

400 Bad Request

Example response body:

```
{  
  "message": "Price must be greater than 0"  
}
```

Another Failure Example — Product Not Found

Request:

GET /api/v1/products/999

Response:

404 Not Found

JSON:

```
{  
  "message": "Product not found"  
}
```

OpenAPI Annotation Example

You can document your Spring Boot endpoint using annotations.

```
@Operation(  
    summary = "Create a product",  
    description = "Creates a new product"  
)  
  
@ApiResponses({  
    @ApiResponse(  
        responseCode = "201",  
        description = "Product created successfully"  
    ),  
    @ApiResponse(  
        responseCode = "400",  
        description = "Invalid product data"  
    ),  
    @ApiResponse(  
        responseCode = "409",
```

```

        description = "Product already exists"
    ),
    @ApiResponse(
        responseCode = "500",
        description = "Internal server error"
    )
})

@PostMapping
public ResponseEntity<Product> createProduct(
    @RequestBody Product product) {

    Product savedProduct =
        productService.createProduct(product);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(savedProduct);
}

```

Step 8 — Organize APIs

Products

This group contains the basic product operations.

For example:

POST /api/v1/products

GET /api/v1/products

GET /api/v1/products/{id}

These APIs are used to create and retrieve products.

Example

Products

└— Create Product

└— Get All Products

└— Get Product by ID

Product Search

This group contains APIs used to search or filter products.

For example:

GET /api/v1/products/search?name=Laptop

Or:

GET /api/v1/products?category=Mobile

You can have search options such as:

Name

Category

Price

Stock

Example:

GET /api/v1/products/search?name=iPhone

The API returns products matching "iPhone".

Product Administration

This group contains APIs used for administrative operations.

For example:

PUT /api/v1/products/{id}

PATCH /api/v1/products/{id}

DELETE /api/v1/products/{id}

These operations modify or remove products.

How to Group APIs in Spring Boot

You can use the @Tag annotation.

First import:

```
import io.swagger.v3.oas.annotations.tags.Tag;
```

Then:

```
@RestController
```

```
@RequestMapping("/api/v1/products")
```

```
@Tag(name = "Products", description = "Product management APIs")
```

```
public class ProductController {
```

```
    // APIs
```

```
}
```

For example, you can create a separate search controller:

```
@RestController
```

```
@RequestMapping("/api/v1/products/search")
```

```
@Tag(name = "Product Search",
```

```
    description = "APIs for searching products")
```

```
public class ProductSearchController {
```

```
    @GetMapping
```

```
    public List<Product> searchProducts(
```

```
        @RequestParam String name) {
```

```
        return productService.searchProducts(name);
```

```
    }
```

```
}
```

And administration:

```

@RestController
@RequestMapping("/api/v1/products")
@Tag(name = "Product Administration",
      description = "APIs for product administration")
public class ProductAdminController {

    @PutMapping("/{id}")
    public Product updateProduct(
        @PathVariable Long id,
        @RequestBody Product product) {
        return productService.updateProduct(id, product);
    }

    @DeleteMapping("/{id}")
    public void deleteProduct(
        @PathVariable Long id) {
        productService.deleteProduct(id);
    }
}

```

Swagger UI will use these @Tag names to organize the endpoints.

Step 9 — Test Through Swagger UI

Start your Spring Boot application

Run your application.

For example:

Product Service

Port: 8083

Your application should start successfully.

Open Swagger UI

Open:

<http://localhost:8083/swagger-ui/index.html>

You should see your API documentation.

For example:

Products

POST /api/v1/products

GET /api/v1/products

GET /api/v1/products/{id}

Test POST

Expand:

POST /api/v1/products

Click:

Try it out

Enter:

```
{  
  "name": "iPhone 17",  
  "description": "Smartphone",  
  "price": 80000,  
  "category": "Mobile",  
  "stock": 25  
}
```

Click:

Execute

Swagger sends the request to your Spring Boot API.

Verify the Request

To verify the **Request**.

Check that Swagger sent:

POST /api/v1/products

with the correct JSON:

```
{  
  "name": "iPhone 17",  
  "description": "Smartphone",  
  "price": 80000,  
  "category": "Mobile",  
  "stock": 25  
}
```

Make sure:

- URL is correct
- HTTP method is correct
- JSON fields are correct
- Values are correct

Verify the Response

After clicking **Execute**, Swagger shows the server response.

For example:

```
{  
  "id": 1,  
  "name": "iPhone 17",  
  "description": "Smartphone",  
  "price": 80000,  
  "category": "Mobile",
```

```
"stock": 25  
}
```

Check that the response contains the expected data.

Verify Status Code

Swagger also shows the HTTP status code.

For successful product creation:

201 Created

For getting a product:

200 OK

For deleting a product:

204 No Content

For an invalid request:

400 Bad Request

For a product that doesn't exist:

404 Not Found

Verify Validation

Try sending invalid data.

For example:

```
{  
  "name": "",  
  "price": -500,  
  "stock": -10  
}
```

Your validation should reject it.

You may get:

400 Bad Request

This confirms that your validation is working.

Verify Error Response

Try requesting a product that doesn't exist:

GET /api/v1/products/9999

If product 9999 doesn't exist, you should receive:

404 Not Found

Example:

```
{  
  "message": "Product not found"  
}
```

This confirms that your error handling works correctly.

Test All APIs

You should test every endpoint.

Method	Endpoint	What to verify
POST	/api/v1/products	Create product
GET	/api/v1/products	Get all products
GET	/api/v1/products/{id}	Get one product
PUT	/api/v1/products/{id}	Update product
PATCH	/api/v1/products/{id}	Partial update
DELETE	/api/v1/products/{id}	Delete product

Step 10 — Final API Review

Naming Standards

Make sure your API names are consistent and meaningful.

Good

GET /api/v1/products

GET /api/v1/products/{id}

POST /api/v1/products

Avoid

GET /api/getAllProducts

POST /api/createProduct

GET /api/getProductById

REST APIs generally use **resources/nouns** rather than action names.

Recommended

/products

/users

/orders

Instead of:

/getProducts

/createProduct

/deleteProduct

Check

Your Product API should consistently use:

/api/v1/products

HTTP Methods

Check whether you are using the correct HTTP method for each operation.

Method Purpose

GET Read data

POST Create data

PUT Complete update

PATCH Partial update

Method Purpose

DELETE Delete data

For your project:

POST /api/v1/products

GET /api/v1/products

GET /api/v1/products/{id}

PUT /api/v1/products/{id}

PATCH /api/v1/products/{id}

DELETE /api/v1/products/{id}

Simple memory trick

POST → Create

GET → Read

PUT → Update

PATCH → Partial Update

DELETE → Delete

Status Codes

Check that your API returns the correct HTTP status codes.

For example:

Successful GET

200 OK

Successful POST

201 Created

Successful DELETE

204 No Content

Invalid request

400 Bad Request

Product not found

404 Not Found

Duplicate product

409 Conflict

Server problem

500 Internal Server Error

Your API should not return 200 OK for every situation.

DTOs

DTO means:

Data Transfer Object

DTOs are objects used to transfer data between the client and server.

Instead of directly exposing your database entity, you can use separate request/response DTOs.

Example

```
public class ProductRequest {  
    private String name;  
    private Double price;  
    private String category;  
    private Integer stock;  
}
```

And:

```
public class ProductResponse {  
    private Long id;  
    private String name;  
    private Double price;  
    private String category;  
    private Integer stock;
```

```
}
```

Then:

Client



ProductRequest DTO



Controller



Service



Database



ProductResponse DTO



Client

Validation

Validation checks whether incoming data is correct.

Suppose you create a product:

```
{  
  "name": "Laptop",  
  "price": 55000,  
  "stock": 10  
}
```

You may require:

name → Cannot be empty

price → Must be greater than 0

stock → Cannot be negative

Example:

```
public class ProductRequest {  
    @NotBlank  
    private String name;  
    @NotNull  
    @Positive  
    private Double price;  
    @NotNull  
    @PositiveOrZero  
    private Integer stock;  
}
```

Then:

@PostMapping

```
public ProductResponse createProduct(  
    @Valid @RequestBody ProductRequest request) {  
    return productService.createProduct(request);  
}
```

Important:

@Valid

tells Spring Boot to perform validation.

Error Handling

Your API should handle errors properly.

For example:

GET /api/v1/products/999

If product 999 doesn't exist, don't simply return an unexpected Java exception.

Return:

404 Not Found

with a useful response:

```
{  
  "message": "Product not found",  
  "status": 404  
}
```

A common Spring Boot approach is:

@RestControllerAdvice

```
public class GlobalExceptionHandler {  
    @ExceptionHandler(ProductNotFoundException.class)  
    public ResponseEntity<?> handleProductNotFound(  
        ProductNotFoundException ex) {  
        return ResponseEntity  
            .status(HttpStatus.NOT_FOUND)  
            .body(Map.of(  
                "message", ex.getMessage(),  
                "status", 404  
            ));  
    }  
}
```

Instead of having error-handling code repeated in every controller, you can handle errors centrally.

Pagination

Pagination is used when there are **many products**.

Imagine you have:

10,000 products

You don't want to return all 10,000 products in one response.

Instead:

GET /api/v1/products?page=0&size=10

This means:

page = 0

size = 10

Return only 10 products.

Example response:

```
{
  "content": [
    {
      "id": 1,
      "name": "Laptop",
      "price": 55000
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 1000,
  "totalPages": 100
}
```

Check

Make sure your API supports pagination where a large dataset is expected.

Sorting

Sorting means arranging products according to a field.

For example:

```
GET /api/v1/products?sort=price,asc
```

Means:

Sort products by price from low to high.

Or:

```
GET /api/v1/products?sort=price,desc
```

Means:

Sort products by price from high to low.

Other examples:

```
sort=name,asc
```

```
sort=name,desc
```

```
sort=price,asc
```

```
sort=stock,desc
```

Filtering

Filtering means retrieving only products that match a condition.

Example:

```
GET /api/v1/products?category=Mobile
```

Returns only mobile products.

Another example:

```
GET /api/v1/products?minPrice=10000&maxPrice=50000
```

Returns products between ₹10,000 and ₹50,000.

API Versioning

API versioning allows you to change your API without immediately breaking existing clients.

Your project uses:

/api/v1/products

Here:

v1

means **Version 1**.

Later you might create:

/api/v2/products

For example:

V1

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 55000  
}
```

V2

```
{  
  "id": 1,  
  "productName": "Laptop",  
  "price": 55000,  
  "category": "Electronics"  
}
```

Existing clients can continue using:

/api/v1/products

while new clients can use:

/api/v2/products

Swagger Documentation

Finally, check that your API is properly documented in Swagger/OpenAPI.

Swagger should show:

Products

POST /api/v1/products

GET /api/v1/products

GET /api/v1/products/{id}

PUT /api/v1/products/{id}

PATCH /api/v1/products/{id}

DELETE /api/v1/products/{id}

And you should be able to click:

Try it out



Enter data



Execute



Check response